6 VALUTARE UNA SOLUZIONE

OBIETTIVI

- Discutere le principali misure di qualità per valutare l'efficacia di una soluzione, tra cui:
    - **Correttezza** basata su metodi empirici.
    - **Efficienza**, esaminando la complessità in termini di tempo e spazio.
    - **Eleganza**, ovvero la massimizzazione di efficacia e semplicità.
    - **Usabilità**, secondo standard di settore.
- Spiegare l'importanza dei compromessi tra le diverse misure di qualità.

VALUTAZIONE DI UNA SOLUZIONE

Hai analizzato il problema, lo hai suddiviso in parti, hai progettato una soluzione, l'hai implementata, testata e corretta. Ora hai qualcosa che funziona. Quindi, hai finito, giusto?

Non proprio.

Avere una soluzione funzionante non significa automaticamente che sia una buona soluzione. Prima di dichiarare il lavoro concluso, devi valutarne la qualità.

Domande chiave per valutare una soluzione

1. **È corretta?** Risolve davvero il problema iniziale?
2. **È efficiente?** Usa le risorse in modo ragionevole?
3. **È elegante?** È semplice ed efficace allo stesso tempo?
4. **È usabile?** È facile da usare per il pubblico a cui è destinata?

Questo capitolo esplora come rispondere a queste domande per valutare la qualità di una soluzione.

È CORRETTA?

La prima e più importante domanda è: **la soluzione risolve effettivamente il problema?**

Se la risposta è *no*, ogni altra misura di qualità diventa irrilevante. Non importa quanto la tua soluzione sia veloce o ben progettata: se non risolve il problema, è **sbagliata** e va corretta.

👉 **Approccio prudente**: **considera un programma errato fino a prova contraria.**

Nella pratica, la correttezza di un programma si verifica tramite **test empirici** (vedi Capitolo 5). Ogni requisito specificato all'inizio dello sviluppo viene trasformato in un test che verifica se il comportamento del programma è corretto.

💡 **Esempio di piano di test per un distributore automatico**

Test #	Oggetto del test	Descrizione	Risultato atteso	Risultato effettivo
1	Slot per monete	Inserire una moneta da 50 cent	Moneta accettata	Moneta accettata
2	Slot per monete	Inserire una moneta straniera	Moneta rifiutata con messaggio	Moneta rifiutata, ma senza messaggio
3	Tastierino	Inserire la selezione valida '15'	"Hai selezionato cioccolata calda"	"Hai selezionato cioccolata calda"
4	Tastierino	Inserire selezione non valida '99'	"Selezione non valida, riprova"	"Selezione non valida, riprova"

Un singolo test fallito è sufficiente a dimostrare che una soluzione è **errata**. Tuttavia, **non è possibile testare tutti i casi possibili**, quindi ogni test superato fornisce solo una maggiore fiducia nella correttezza.

Livelli di correttezza (dal meno al più convincente)

1. Nessun errore di sintassi o operazioni invalide.
2. La soluzione funziona per un set limitato di dati di test.
3. Funziona per dati casuali o tipici.
4. Funziona per dati difficili da elaborare.
5. Funziona per tutti i dati validi.
6. Funziona anche per dati errati, restituendo un messaggio adeguato.
7. Funziona correttamente per **qualsiasi** input.

👉 **Conclusione**: **Una soluzione può essere corretta senza essere efficiente o elegante.**

È EFFICIENTE?

Tutte le soluzioni richiedono risorse per funzionare. Alcuni algoritmi, pur risolvendo lo stesso problema, **sono più efficienti di altri**.

L'efficienza si misura in termini di:

- **Tempo**: quanto impiega l'algoritmo per essere eseguito.
- **Spazio**: quanta memoria utilizza l'algoritmo.

👉 **Concetto chiave: complessità computazionale**
L'efficienza di un algoritmo si classifica tramite la **notazione Big-O**, che indica la relazione tra tempo di esecuzione e dimensione dell'input.

Esempi di classi di complessità (Big-O)

Nome	Tempo di esecuzione	Esempio
O(1) - Costante	Tempo fisso	Controllare se un numero è pari o dispari
O(logN) - Logaritmica	Cresce lentamente	Ricerca binaria
O(N) - Lineare	Cresce proporzionalmente	Scorrere una lista
O(N^2) - Quadratica	Peggiora rapidamente	Bubble Sort
O(N!) - Fattoriale	Esplosione combinatoria	Forza bruta su combinazioni

💡 **Esempio: Ricerca lineare**
Dato un elenco di N elementi, cerchiamo un valore specifico:

- **Caso migliore**: lo troviamo al primo tentativo → O(1).
- **Caso medio**: lo troviamo a metà lista → O(N/2) → O(N).
- **Caso peggiore**: non è presente → O(N).

👉 **Conclusione**: le soluzioni con **O(N), O(logN) o meglio** sono considerate **efficienti**. Quelle con **O(N!) o O(2^N)** diventano rapidamente inutilizzabili.

È ELEGANTE?

Eleganza = **Massima efficacia con la minima complessità.**

Una soluzione elegante è:

- **Chiara**: facile da capire e mantenere.
- **Semplice**: risolve più problemi con meno codice.
- **Non ovvia**: a volte richiede creatività per essere scoperta.

💡 **Esempio: Somma dei numeri da 1 a 100**

Soluzione ovvia:

pseudocode

CopiaModifica

somma = 0

per ogni numero n da 1 a 100:

  somma = somma + n

👉 **O(N) - 100 operazioni**

Soluzione elegante di Gauss:

pseudocode

CopiaModifica

somma = (100 * 101) / 2

👉 **O(1) - Solo 1 operazione!**

❗ **Attenzione**: soluzioni troppo "furbe" possono essere difficili da capire per altri (o per te stesso in futuro!).

È USABILE?

Una soluzione efficace deve essere **facile e piacevole da usare**.

📌 **Misure di usabilità (Nielsen, 2012):**

1. **Facilità di apprendimento**
2. **Efficienza d'uso**
3. **Memorizzabilità**
4. **Gestione degli errori**
5. **Soddisfazione dell'utente**

💡 **Test di usabilità**

- Osservare gli utenti mentre usano la soluzione.
- Far compilare un questionario per raccogliere feedback.

COMPROMESSI

🔀 **Esempi di trade-off**

- **Compressione immagini**: meno spazio occupato, ma più tempo per decomprimerle.

- **Codice elegante vs leggibile**: una soluzione compatta può essere meno comprensibile.

👉 **Conclusione**: **non si può ottimizzare tutto!** Devi decidere quali aspetti sono prioritari.